

CS456 Design and Analysis of Algorithms

Project 3 - Counting Inversions

Deepshan Adhikari (800846035)
deepadh@siue.edu

1. Introduction

An inversion is a pair of elements in an array that are in the wrong order. If i comes before j but $A[i]$ is bigger than $A[j]$, then (i, j) is one inversion. For example, the array 3, 1, 4, 2 has three inversions: (3, 1), (3, 2), and (4, 2). Inversions are useful because they tell us how far an array is from being sorted. A sorted array has zero inversions. A reverse-sorted array has the maximum possible. This measure is used in recommendation systems to compare rankings, in genomics to compare gene sequences, and in algorithm analysis to predict how long certain sorts will take. Inversions matter for Insertion Sort because every shift that Insertion Sort performs removes exactly one inversion from the array.

1.1 Algorithms Implemented

Three algorithms were implemented.

a. Naive Inversion Counting: This method checks every pair of elements and counts how many are out of order. It is the simplest approach and is used to check that the faster method gives the right answer.

b. Divide-and-Conquer Inversion Counting: This is a modified MergeSort from Kleinberg and Tardos [1], Section 5.3. The array is split in half, each half is sorted with the same method, and inversions between the two halves are counted during the merge. When an element from the right half is placed before the remaining elements of the left half, every remaining left element is larger than it, and each one forms an inversion.

c. Insertion Sort: This is the standard Insertion Sort. It walks through the array and shifts each element left past every larger element already in the sorted prefix. Since each shift removes one inversion, the total number of shifts performed by Insertion Sort equals the number of inversions in the input. However, the algorithm also scans elements that do not require shifting, so its total running time is $O(n+I)$, where n is the input size and I is the inversion count.

1.2 Time Complexity

The Naive method runs in $O(n^2)$ time because it checks every pair. The Divide-and-Conquer method runs in $O(n \log n)$ time, the same as MergeSort, since counting adds only a constant amount of work per merge. Insertion Sort runs in $O(n + I)$ time, where I is the number of inversions.

Algorithms	Best Case	Average Case	Worst Case
Naive Inversion Counting	$O(n^2)$	$O(n^2)$	$O(n^2)$
Divide-and-Conquer Counting	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$

1.3 Implementation Details

The code was written in C++. Two experiments were run.

Experiment 1: Inversions and Sort Time Across All Dataset Types and Sizes

Three dataset types were tested: Sorted, Random, and Reverse Sorted. Five sizes were used: 10,000, 20,000, 50,000, 100,000, and 200,000 elements. For each array, inversions were counted with both the naive and the divide-and-conquer methods, and the two counts were checked to be equal. Insertion Sort was then timed. Every configuration was run 10 times and the average was recorded.

Experiment 2: Effect of Increasing Disorder on Sort Time ($n = 100,000$)

This experiment corresponds to the additional investigation required for graduate students. Starting with a sorted array of size 100,000, random swaps were applied at five levels: 1%, 5%, 10%, 25%, and 50% of the array size. For example, at the 1% level, $100,000 * 0.01 = 1,000$ swaps were performed. Each swap picks two random indices and exchanges the values. The percentage refers to the number of swaps, not the number of inversions, so the two are not directly proportional. For each level, the inversion count and the Insertion Sort time were recorded, averaged over 10 trials. Time is recorded in milliseconds per trial.

2. Results

2.1 Experiment 1: Inversions and Sort Time Across All Dataset Types and Sizes

Figure 1 shows two things for each array: how many inversions it had, and how long Insertion Sort took to sort it. Each time is the average of 10 runs. Time is recorded in milliseconds per trial.

Experiment 1: Inversions and Sort Time Across All Dataset Types and Sizes			
Array Size	Dataset Type	Inversion Count	Average Insertion Sort Time (ms)
10,000	Sorted	0	0.0050
10,000	Random	24975835	6.0751
10,000	Reverse Sorted	49995000	12.1160
20,000	Sorted	0	0.0063
20,000	Random	99743900	24.2566
20,000	Reverse Sorted	199990000	48.5599
50,000	Sorted	0	0.0157
50,000	Random	624115765	153.3427
50,000	Reverse Sorted	1249975000	309.4103
100,000	Sorted	0	0.0320
100,000	Random	2497316064	623.6994
100,000	Reverse Sorted	4999950000	1242.8737
200,000	Sorted	0	0.0644
200,000	Random	10001760621	2576.7234
200,000	Reverse Sorted	19999900000	4975.7102

Figure 1: Inversion Count and Average Insertion Sort Time for All Dataset Types and Sizes.

The Sorted arrays contained zero inversions and sorted almost instantly. Their times grew from 0.0050 ms at 10,000 elements to 0.0644 ms at 200,000 elements. Because the array is already in order, Insertion Sort only needs to scan through it once.

Random arrays had about $n^2/4$ inversions, and reverse-sorted arrays had the maximum possible $n(n-1)/2$ inversions. Because of this, both took much longer than sorted arrays. Reverse Sorted was always about twice as slow as Random, since it has about twice as many inversions. At $n = 200,000$ the gap was very large. The Sorted array took 0.0644 ms, while the Reverse Sorted array took 4,975.7102 ms. That is really slower, even though both arrays have the same length.

Sorted times increased by a factor of about two each time the array size doubled, which matches the $O(n)$ behavior. Random and Reverse Sorted times increased by a factor of about four over the same size increase, which matches the $O(n^2)$ behavior.

The table therefore shows that Insertion Sort time depends on both the array size and the number of inversions it contains, not on size alone.

2.2 Experiment 2: Effect of Increasing Disorder on Sort Time (n = 100,000)

Figure 2 reports how the amount of disorder in a fixed-size array of 100,000 elements affects both the inversion count and the time Insertion Sort takes to sort it. Each time value is the average of 10 runs. Time is recorded in milliseconds per trial.

Experiment 2: Effect of Increasing Disorder on Sort Time (n = 100,000)		
Swap %	Inversion Count	Average Insertion Sort Time (ms)
1%	65763636	17.0898
5%	308973391	78.2087
10%	579443814	167.3136
25%	1180900301	303.9001
50%	1771633117	447.6027

Figure 2: Inversion Count and Insertion Sort Time Across Disorder Levels (n = 100,000).

At the 1% level, where 1,000 random swaps were applied to an initially sorted array, the array contained about 65.8 million inversions, and Insertion Sort took 17.0898 ms. At the 5% level, with 5,000 swaps, the inversion count rose to about 309.0 million and the time rose to 78.2087 ms. The pattern continued at the 10% and 25% levels, with inversion counts of about 579.4 million and 1.18 billion, and times of 167.3136 ms and 303.9001 ms.

At the 50% level, with 50,000 swaps, the array contained about 1.77 billion inversions, and Insertion Sort took 447.6027 ms. This is the most disordered case tested, and it also had the largest inversion count and the slowest sort time. The swap percentage refers to the number of swaps as a fraction of the array size, not to the fraction of maximum possible inversions. Even at 50%, the inversion count is well below the maximum of about 5 billion for $n = 100,000$, because many random swaps cancel each other out.

2.3 Charts

Insertion Sort Time vs. Array Size

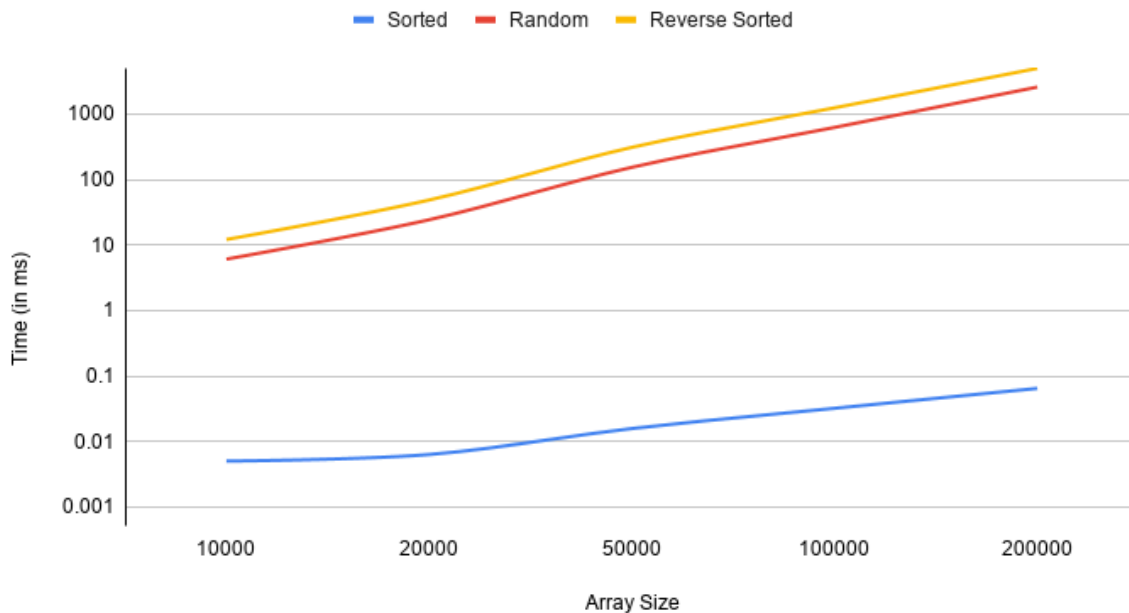


Figure 3: Insertion Sort Time (in ms) vs. Array Size.

Figure 3 plots the average Insertion Sort time against array size for the Sorted, Random, and Reverse Sorted datasets. The y-axis, which represents time in milliseconds, uses a logarithmic scale so that both the small values for Sorted arrays and the large values for Reverse Sorted arrays are visible. The Sorted line stays very low because a sorted array has no inversions. The Random and Reverse Sorted lines rise much faster as array size increases because those arrays contain many inversions. The Reverse Sorted line is above the Random line at every size, consistent with reverse-sorted arrays having about twice as many inversions as random arrays.

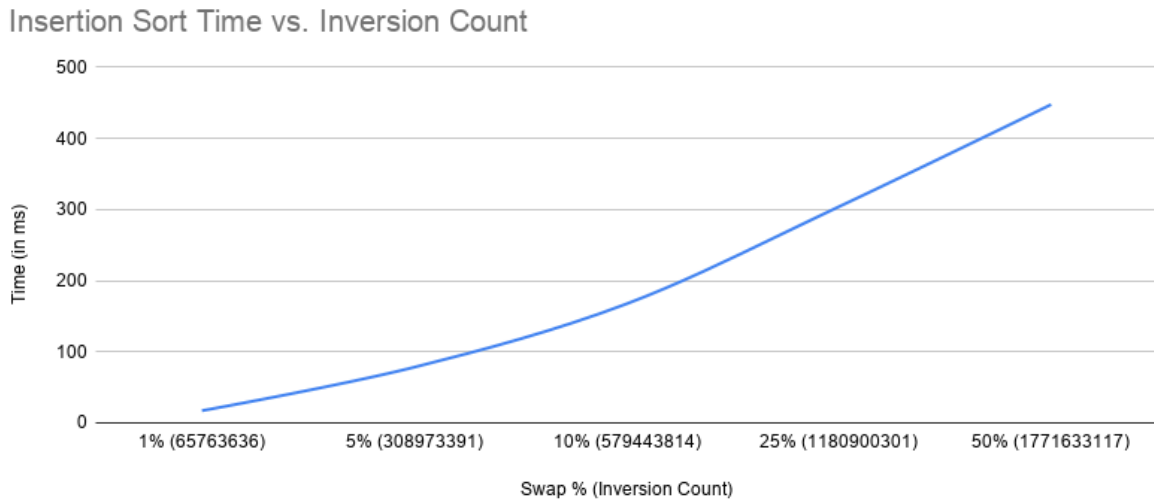


Figure 4: Insertion Sort Time vs Inversion Count Across Disorder Levels ($n = 100,000$).

Figure 4 shows the average Insertion Sort time for five swap levels at a fixed array size of $n=100,000$. Each category is labeled with its swap percentage and resulting inversion count. The chart shows that insertion-sort time increased as the swap level and inversion count increased. These results suggest that inversion count can help predict Insertion Sort performance for arrays of this size.

3. Discussion

3.1 Which datasets had the most and fewest inversions

The Reverse Sorted arrays had the most inversions at every size. A reverse-sorted array of size n contains exactly $n(n-1)/2$ inversions, which is the maximum possible. At $n = 200,000$ this equals 19,999,900,000 inversions. The Random arrays came next, with about half as many. The Sorted arrays had the fewest, with zero inversions at every size because no pair is out of order.

3.2 How inversion count affected Insertion Sort performance

The effect was large and consistent. At $n = 200,000$, the Sorted array took 0.0644 ms while the Reverse Sorted array took 4,975.7102 ms. Both arrays have the same length, so the only difference is how many inversions they contain. The Random array took 2,576.7234 ms, sitting about halfway between the two extremes. This is expected, because a random array contains roughly half the inversions of a reverse-sorted one.

The measured times matched the predicted behavior. Within a fixed size, time scaled with the inversion count. Across sizes, when the array size doubled, Sorted times roughly doubled, which matches $O(n)$, while Random and Reverse Sorted times roughly quadrupled, which matches $O(n^2)$. Going from $n = 50,000$ to $n = 100,000$ on the Reverse Sorted dataset, both the inversion count and the running time grew by a factor of about four, exactly what a quadratic algorithm predicts. The same pattern held from 100,000 to 200,000, where the time grew from 1,242.8737 ms to 4,975.7102 ms, again close to a factor of four.

This confirms that Insertion Sort does not depend on size alone. Two arrays of the same length can have very different running times if they contain different numbers of inversions.

3.3 Can inversion count predict Insertion Sort performance

Yes. Experiment 2 gives direct evidence for this. At a fixed size of 100,000, the five disorder levels produced very different inversion counts, from about 65.8 million at 1% swaps to about 1.77 billion at 50% swaps. The Insertion Sort times scaled with the inversion counts in the same way: 17.0898 ms at 1%, 78.2087 ms at 5%, 167.3136 ms at 10%, 303.9001 ms at 25%, and 447.6027 ms at 50%.

When each time is divided by its inversion count, the result is almost the same at every level. The five values stay close to a constant, which means the cost per inversion does not change as the array becomes more disordered. If the runtime depended on something other than inversion count, this value would drift as the swap percentage increased. It does not.

This shows that Insertion Sort runtime is proportional to the number of inversions in the input, at least across the range of disorder tested here. Inversion count is therefore a reliable predictor of Insertion Sort performance at a fixed array size.

4. Source Code

```
// CS456 Design and Analysis of Algorithms
// Project 3 - Counting Inversions

// This program implements three algorithms:
// 1. Naive Inversion Counting - O(n^2)
// 2. Divide-and-Conquer Count - O(n log n), modified MergeSort
// 3. Insertion Sort - O(n^2), timed

// Experiment 1 tests Sorted, Random, and Reverse Sorted arrays at five sizes (10k,
// 20k, 50k, 100k, 200k).
// Each configuration runs 10 times.
// The reported Insertion Sort time is the average over 10 trials.

// Experiment 2 starts from a sorted array and applies random swaps at
// 1%, 5%, 10%, 25%, and 50% levels on a fixed size of 100,000. Time per
// inversion is reported in nanoseconds to test whether inversion count
// predicts Insertion Sort runtime.

#include <iostream> // reads input from the console and prints output (cout, cerr)
#include <vector>    // stores the arrays of numbers that will be sorted and
counted
#include <string>    // stores text such as "Sorted", "Random", and "Reverse
Sorted"
#include <numeric>   // provides iota, which fills an array with 0, 1, 2, ... in
order
#include <algorithm> // provides shuffle and swap, used to build random and
partially sorted arrays
#include <random>    // provides the random number generators used to build random
arrays
#include <chrono>    // provides the high-resolution timer used to measure
Insertion Sort
#include <iomanip>    // provides formatting tools (fixed, setprecision, setw) for
the output tables

using namespace std;
using namespace chrono;

// Shared random generator, seeded from the operating system.
// Each call advances the sequence, so random arrays differ between calls.
static mt19937 randomGenerator(random_device{}());

// Counts inversions by checking every pair (i, j) with i < j and A[i] > A[j].
// O(n^2). Used to verify the divide-and-conquer count.
```

```

long long naiveCount(const vector<int> &values)
{
    long long inversions = 0;
    int size = (int)values.size();
    for (int i = 0; i < size; i++)
    {
        for (int j = i + 1; j < size; j++)
        {
            if (values[i] > values[j])
                inversions++;
        }
    }
    return inversions;
}

// Merges two sorted halves and counts cross inversions.
// When a right-half element is chosen, every remaining left-half element
// is larger than it, so each one forms an inversion.
long long mergeAndCount(vector<int> &values, vector<int> &buffer,
                        int low, int mid, int high)
{
    int leftIndex = low;
    int rightIndex = mid + 1;
    int bufferIndex = low;
    long long inversions = 0;

    while (leftIndex <= mid && rightIndex <= high)
    {
        if (values[leftIndex] <= values[rightIndex])
        {
            buffer[bufferIndex++] = values[leftIndex++];
        }
        else
        {
            buffer[bufferIndex++] = values[rightIndex++];
            inversions += (mid - leftIndex + 1);
        }
    }

    while (leftIndex <= mid)
        buffer[bufferIndex++] = values[leftIndex++];
    while (rightIndex <= high)
        buffer[bufferIndex++] = values[rightIndex++];

    for (int index = low; index <= high; index++)

```

```

        values[index] = buffer[index];

        return inversions;
    }

    // Recursively counts inversions in the left half, the right half,
    // and across the two halves during the merge step.
    long long countInversionsRecursive(vector<int> &values, vector<int> &buffer,
                                      int low, int high)
    {
        if (low >= high)
            return 0;

        int mid = low + (high - low) / 2;
        long long inversions = 0;

        inversions += countInversionsRecursive(values, buffer, low, mid);
        inversions += countInversionsRecursive(values, buffer, mid + 1, high);
        inversions += mergeAndCount(values, buffer, low, mid, high);

        return inversions;
    }

    // Wrapper that prepares the buffer and starts the recursion.
    // The array is passed by value because the recursion rearranges it.
    long long countInversions(vector<int> values)
    {
        vector<int> buffer(values.size());
        return countInversionsRecursive(values, buffer, 0, (int)values.size() - 1);
    }

    // Standard Insertion Sort. Returns elapsed time in milliseconds.
    // The number of shifts equals the number of inversions in the input,
    // so runtime is proportional to the inversion count.
    double insertionSort(vector<int> values)
    {
        auto startTime = high_resolution_clock::now();

        int size = (int)values.size();
        for (int i = 1; i < size; i++)
        {
            int key = values[i];
            int j = i - 1;
            while (j >= 0 && values[j] > key)
            {

```

```

        values[j + 1] = values[j];
        j--;
    }
    values[j + 1] = key;
}

auto endTime = high_resolution_clock::now();
return duration<double, milli>(endTime - startTime).count();
}

// Creates a sorted array of values 0 to size-1.
vector<int> makeSortedArray(int size)
{
    vector<int> values(size);
    iota(values.begin(), values.end(), 0);
    return values;
}

// Creates a randomly shuffled array of values 0 to size-1.
vector<int> makeRandomArray(int size)
{
    vector<int> values = makeSortedArray(size);
    shuffle(values.begin(), values.end(), randomGenerator);
    return values;
}

// Creates a reverse-sorted array from size down to 1.
// Has the maximum possible inversions, size*(size-1)/2.
vector<int> makeReverseArray(int size)
{
    vector<int> values(size);
    for (int i = 0; i < size; i++)
        values[i] = size - i;
    return values;
}

// Starts from a sorted array and performs size * swapPercent random swaps.
vector<int> makePartiallySortedArray(int size, double swapPercent)
{
    vector<int> values = makeSortedArray(size);
    int swapCount = (int)(size * swapPercent);
    uniform_int_distribution<int> indexPicker(0, size - 1);

    for (int s = 0; s < swapCount; s++)
    {

```

```

        int firstIndex = indexPicker(randomGenerator);
        int secondIndex = indexPicker(randomGenerator);
        swap(values[firstIndex], values[secondIndex]);
    }
    return values;
}

// Holds one row of results for the final summary.
struct ResultRow
{
    long long size;          // array size, or swap percentage for Experiment 2
    string label;            // dataset label, e.g. "Sorted" or "10%"
    long long inversions;    // average inversion count
    double timeMs;          // average Insertion Sort time in milliseconds
};

// Formats a number with commas, e.g. 10000 becomes "10,000".
string formatWithCommas(long long number)
{
    string text = to_string(number);
    for (int position = (int)text.size() - 3; position > 0; position -= 3)
        text.insert(position, ",");
    return text;
}

// Prints every row that matches the given extreme value.
// Shows all ties rather than only the first one.
void printAllTies(const vector<ResultRow> &rows,
                 bool matchOnTime,
                 long long inversionTarget,
                 double timeTarget)
{
    bool firstMatch = true;
    for (const ResultRow &row : rows)
    {
        bool isMatch = matchOnTime
                        ? (row.timeMs == timeTarget)
                        : (row.inversions == inversionTarget);

        if (isMatch)
        {
            if (!firstMatch)
                cout << ", ";

            cout << row.label << " n=" << formatWithCommas(row.size);
            firstMatch = false;
        }
    }
}

```

```

    }

    cout << "\n";
}

int main()
{
    const int TRIALS = 10;

    // Sizes and dataset types for Experiment 1.
    vector<int> arraySizes = {10000, 20000, 50000, 100000, 200000};
    vector<string> datasetTypes = {"Sorted", "Random", "Reverse"};

    // Largest size where the naive count is used to verify the Divide-and-Conquer
    count.

    // Set to 200,000 so every array is verified, as the assignment requires.
    const int VERIFICATION_LIMIT = 200000;

    // Column widths for the console tables.
    const int SIZE_COLUMN_WIDTH = 12;
    const int TYPE_COLUMN_WIDTH = 16;
    const int INVERSION_COLUMN_WIDTH = 16;
    const int TIME_COLUMN_WIDTH = 36;

    // Separator line lengths, derived from the column widths.
    const int EXPERIMENT1_LINE_LENGTH =
        SIZE_COLUMN_WIDTH + TYPE_COLUMN_WIDTH + INVERSION_COLUMN_WIDTH +
    TIME_COLUMN_WIDTH + 9;
    const int EXPERIMENT2_LINE_LENGTH =
        SIZE_COLUMN_WIDTH + INVERSION_COLUMN_WIDTH + TIME_COLUMN_WIDTH + 6;

    vector<ResultRow> sweepRows;    // Experiment 1 results
    vector<ResultRow> disorderRows; // Experiment 2 results

    // Experiment 1 header.
    cout << "\nProject 3 - Counting Inversions\n";
    cout << "\nExperiment 1: Inversions and Sort Time Across All Dataset Types and
    Sizes\n";
    cout << string(EXPERIMENT1_LINE_LENGTH, '-') << "\n";
    cout << left
        << setw(SIZE_COLUMN_WIDTH) << "Array Size" << " | "
        << setw(TYPE_COLUMN_WIDTH) << "Dataset Type" << " | "
        << setw(INVERSION_COLUMN_WIDTH) << "Inversion Count" << " | "
        << setw(TIME_COLUMN_WIDTH) << "Average Insertion Sort Time (ms)" << "\n";
    cout << string(EXPERIMENT1_LINE_LENGTH, '-') << "\n";

```

```

// Experiment 1: for each size and dataset type, count inversions with
// both methods, verify they match, and time Insertion Sort. Averages
// are taken over 10 trials. A dashed line separates each size group.
for (int size : arraySizes)
{
    for (const string &type : datasetTypes)
    {
        long long inversionSum = 0;
        double timeSum = 0.0;

        for (int trial = 0; trial < TRIALS; trial++)
        {
            vector<int> data;
            if (type == "Sorted")
                data = makeSortedArray(size);
            else if (type == "Random")
                data = makeRandomArray(size);
            else
                data = makeReverseArray(size);

            long long divideAndConquerCount = countInversions(data);

            // Verify against the naive count. The naive method is slow
            // at 200,000, but the assignment requires it to run for
            // every array.
            if (size <= VERIFICATION_LIMIT)
            {
                long long naiveTotal = naiveCount(data);
                if (naiveTotal != divideAndConquerCount)
                {
                    cerr << "MISMATCH at n=" << size << " type=" << type <<
"\n";

                    return 1;
                }
            }

            inversionSum += divideAndConquerCount;
            timeSum += insertionSort(data);
        }

        long long averageInversions = inversionSum / TRIALS;
        double averageTime = timeSum / TRIALS;

        // Print "Reverse Sorted" in full instead of the short label.
        string printedLabel = (type == "Reverse") ? "Reverse Sorted" : type;

```

```

        cout << left
            << setw(SIZE_COLUMN_WIDTH) << formatWithCommas(size) << " | "
            << setw(TYPE_COLUMN_WIDTH) << printedLabel << " | "
            << setw(INVERSION_COLUMN_WIDTH) << averageInversions << " | "
            << fixed << setprecision(4)
            << setw(TIME_COLUMN_WIDTH) << averageTime << "\n";

        sweepRows.push_back({size, type, averageInversions, averageTime});
    }

    cout << string(EXPERIMENT1_LINE_LENGTH, '-') << "\n";
}

    cout << "\nExperiment 2: Effect of Increasing Disorder on Sort Time (n =
100,000)\n";
    cout << string(EXPERIMENT2_LINE_LENGTH, '-') << "\n";
    cout << left
        << setw(SIZE_COLUMN_WIDTH) << "Swap %" << " | "
        << setw(INVERSION_COLUMN_WIDTH) << "Inversion Count" << " | "
        << setw(TIME_COLUMN_WIDTH) << "Average Insertion Sort Time (ms)" << "\n";
    cout << string(EXPERIMENT2_LINE_LENGTH, '-') << "\n";

    vector<double> swapLevels = {0.01, 0.05, 0.10, 0.25, 0.50};
    int fixedSize = 100000;

    for (double swapPercent : swapLevels)
    {
        long long inversionSum = 0;
        double timeSum = 0.0;

        for (int trial = 0; trial < TRIALS; trial++)
        {
            vector<int> data = makePartiallySortedArray(fixedSize, swapPercent);
            inversionSum += countInversions(data);
            timeSum += insertionSort(data);
        }

        long long averageInversions = inversionSum / TRIALS;
        double averageTime = timeSum / TRIALS;

        string percentageLabel = to_string((int)(swapPercent * 100)) + "%";

        cout << left
            << setw(SIZE_COLUMN_WIDTH) << percentageLabel << " | "

```

```

        << setw(INVERSION_COLUMN_WIDTH) << averageInversions << " | "
        << fixed << setprecision(4)
        << setw(TIME_COLUMN_WIDTH) << averageTime << "\n";

        disorderRows.push_back(({(long long)(swapPercent * 100), percentageLabel,
averageInversions, averageTime}));
    }
    cout << string(EXPERIMENT2_LINE_LENGTH, '-') << "\n";

    // Summary of Experiment 1
    // Reports the minimum and maximum inversion counts and times, listing
    // every configuration that ties for the extreme value.
    cout << "\nSummary\n";

    long long fewestInversions = sweepRows[0].inversions;
    long long mostInversions = sweepRows[0].inversions;
    double fastestTime = sweepRows[0].timeMs;
    double slowestTime = sweepRows[0].timeMs;

    for (const ResultRow &row : sweepRows)
    {
        if (row.inversions < fewestInversions)
            fewestInversions = row.inversions;
        if (row.inversions > mostInversions)
            mostInversions = row.inversions;
        if (row.timeMs < fastestTime)
            fastestTime = row.timeMs;
        if (row.timeMs > slowestTime)
            slowestTime = row.timeMs;
    }

    cout << "Experiment 1:\n";
    cout << "  Fewest inversions : " << fewestInversions << " at ";
    printAllTies(sweepRows, false, fewestInversions, 0.0);
    cout << "  Most inversions   : " << mostInversions << " at ";
    printAllTies(sweepRows, false, mostInversions, 0.0);
    cout << "  Fastest sort       : " << fixed << setprecision(4) << fastestTime << "
ms at ";
    printAllTies(sweepRows, true, 0, fastestTime);
    cout << "  Slowest sort        : " << fixed << setprecision(4) << slowestTime << "
ms at ";
    printAllTies(sweepRows, true, 0, slowestTime);

    // Summary of Experiment 2
    // Time per inversion is shown in nanoseconds for easier comparison.

```

```

// Compare the values across disorder levels to assess how well
// inversion count relates to Insertion Sort runtime.
cout << "\nExperiment 2:\n";
cout << "  Time per inversion (in nanoseconds for easier comparison):\n";
for (const ResultRow &row : disorderRows)
{
    double nanosecondsPerInversion = (row.inversions > 0)
                                      ? (row.timeMs * 1e6 / row.inversions)
                                      : 0.0;

    cout << "    " << setw(6) << row.label << " -> "
          << fixed << setprecision(3) << nanosecondsPerInversion
          << " ns per inversion\n";
}

cout << "\nDone.\n";
return 0;
}

```

5. Conclusion

This project compared three algorithms related to inversion counting on three dataset types at five array sizes: 10,000, 20,000, 50,000, 100,000, and 200,000 elements. Two experiments were run. The first experiment tested all three dataset types at all five sizes, and the second experiment tested how increasing disorder affects sort time at a fixed size of 100,000.

The results show that inversion count has a large effect on how long Insertion Sort takes. Sorted arrays contain zero inversions and sort almost instantly. Reverse Sorted arrays contain the maximum possible inversions and take much longer. At $n = 200,000$, the Sorted array took 0.0644 ms while the Reverse Sorted array took 4,975.7102 ms, about 77,000 times slower, even though both arrays have the same length. Random arrays fell between the two extremes, with about half the inversions of a reverse-sorted array and about half the running time.

The growth rates matched the expected behavior. When the array size doubled, Sorted times roughly doubled, matching $O(n)$, while Random and Reverse Sorted times roughly quadrupled, matching $O(n^2)$. This confirms that Insertion Sort depends on both the array size and the number of inversions in the input, not on size alone.

The second experiment showed that at a fixed size of 100,000, the time per inversion stayed almost constant across all five disorder levels. This means Insertion Sort runtime is proportional to inversion count, and inversion count can be used to predict how long the sort will take.

The divide-and-conquer method matched the naive method on every array tested, which confirms that the faster inversion counting algorithm was implemented correctly.

References

[1] Kleinberg, Jon & Tardos, Éva. (2006). Algorithm Design.